

1. Intelligent Agents

AI: study of agents that receive percepts & act on environment. Agent = program + architecture.
PEAS: Performance measure, Environment, Actuators, Sensors. A **rational agent** chooses actions maximizing performance measure given percepts & knowledge.

Task Environment Properties

- **Fully** vs **Partially** observable: sensors give complete state or not
- **Single** vs **Multi-agent**
- **Deterministic** vs **Stochastic**: next state fully determined by current state+action, or involves probabilities
- **Episodic** vs **Sequential**: episodes independent, or current decision affects future
- **Static** vs **Dynamic** (semi-dynamic: world fixed but score changes with time)
- **Discrete** vs **Continuous**: state/time/percepts/actions
- **Known** vs **Unknown**: outcomes of actions given or not

This course: fully-observable, deterministic, static, known only.

Agent Structures

- **Simple reflex**: condition-action rules only
- **Goal-based**: acts to achieve a goal (search-based)
- **Utility-based**: maximizes a utility (score) function
- **Learning agent**: Performance element (acts), Learning element (improves PE), Critic (feedback vs standard), Problem generator (proposes exploration)

Exploration vs exploitation: learn about world vs maximize gain now.

Problem Formulation (systematic search)

- **States**: state-space
- **Initial state**
- **Goal test/state(s)**
- **Actions** $actions(state)$; branching factor b : $|actions(s)| \leq b \forall s$
- **Transition**: $new_state = transition(state, action)$
- **Action cost function**

Steps: Goal Formulation → Problem Formulation → Search → Execute.

2. Search Fundamentals

Path: sequence of actions. **Solution**: path from initial to goal state. **Optimal solution**: lowest path cost. A search algo implicitly builds a **search tree**; a

node holds a state (state may repeat across nodes).
create frontier
insert Node(initial_state)
while frontier not empty:
 node = frontier.pop()
 if node.state is goal: **return solution**
 for action in actions(node.state):
 next = transition(node.state,action)
 frontier.add(Node(next))
return failure

Complete: finds a solution if one exists (for every instance). **Optimal**: every solution it returns is the best possible. (An algo can be optimal and incomplete!)

Uninformed Search (no info on goal distance)

- **BFS**: frontier=queue. Explores layer by layer. Complete & optimal if finite b , goal at finite depth, unit step cost.
- **UCS**: frontier=priority queue by path cost $g(n)$. Expands "tier by tier" of cost. Complete/optimal if all costs $\geq \epsilon > 0$ (complete) / ≥ 0 (optimal).
- **DFS**: frontier=stack. Goes deep, backtracks. Poly space, NOT complete (infinite depth) NOT optimal.

Queue=FIFO=BFS spreads outward evenly;
Stack=LIFO=DFS commits to one path; Priority queue=cheapest-first=UCS.

Graph search variant adds a **visited** set to avoid repeats.

Search Strategies

- **DLS** (depth-limited): DFS with depth cutoff l ; backtrack at limit. Poly space, NOT complete/optimal.
- **IDS** (iterative deepening): run DLS for $l = 0, 1, 2, \dots$ Poly space (like DFS) but complete & optimal (like BFS)! Overhead from re-exploring shallow nodes is acceptable since cost is dominated by deepest layer.

3. Informed Search

Heuristic $h(n)$: estimate of optimal cost from n to a goal. Must be ≥ 0 , $h(goal) = 0$.

Greedy Best-First: $f(n) = h(n)$ only $\Rightarrow$ fast but suboptimal.

A* Search: priority queue on $f(n) = g(n) + h(n)$

$g(n)$ =cost so far, $h(n)$ =heuristic to goal. g trusts the past, h guesses the future – A* balances both.

Admissible: $0 \leq h(n) \leq h^*(n)$ for every n (never overestimates true cost h^*). $\Rightarrow$ A* without visited memory: complete & optimal.

Consistent (triangle inequality):

$h(n) \leq c(n, a, n') + h(n')$. $\Rightarrow$ A* with or without visited memory: complete & optimal. Consistency $\Rightarrow$ Admissibility.

Dominance: if $h_1(n) \geq h_2(n) \forall n$ (both admissible), h_1 dominates $h_2 \Rightarrow h_1$ better (fewer nodes expanded, closer to h^*).

Inventing Heuristics

- **Relaxed problem**: fewer restrictions on actions; its optimal cost is admissible for original. E.g. straight-line distance (drop road constraint).
- **Pattern databases**: precompute exact costs for subproblems.
- **Learning** from data.

Search Algorithms Summary (worst case)

Algo	Time	Space	Compl.?	Opt.?
BFS	Exp	Exp	Yes	Yes
UCS	Exp	Exp	Yes ³	Yes ³
DFS	Exp	Poly	No/Yes ⁴	No
DLS	Exp	Poly ²	No	No ²
IDS	Exp	Poly ²	Yes	Yes
A*	Exp	Exp	Yes ³	if admiss.
A*+mem	Exp	Exp	Yes ³	if consist.

¹ w.r.t. depth/tier ² if used w/ DFS ³ if edge costs ≥ 0 ⁴ with visited memory

4. Local Search

For hard/huge state spaces (TSP, CSPs, continuous opt.) – systematic search intractable (NP-hard).

Local search: no path/reached-set tracking; **solution = a state**. Not systematic $\Rightarrow$ typically incomplete/suboptimal, but low memory, works on huge/infinite spaces.

Successor function: generates neighbouring states (replaces actions/transition). **Evaluation (fitness) function**: $value = eval(state)$; minimize or maximize.

Hill Climbing (steepest ascent): **current** = initial_state

while True:
 best = highest-eval successor of **current**
 if eval(best) <= eval(current): **return current**
 current = best

Always take the best next step – greedy, can get stuck.

Landscape: Global max (best overall); Local max (best among neighbors only); Shoulder (flat region, hard to escape).

5. Adversarial Search

Multi-agent: Competitive (opposing goals) / Cooperative / Mixed-motive. **Two-player**

zero-sum, turn-taking: fully observable, deterministic, discrete, no infinite runs.

Problem Formulation

States, Initial state, **Terminal states** (win/lose/draw), Actions, Transition, **Utility function** $utility(state)$ = value from Player A's perspective (e.g. +1/ – 1/0).

Minimax: assume both play optimally. A maximizes, B minimizes A's outcome. **def** max_value(state):

```
  if is_terminal(state): return utility(state)
  v = -inf
  for s' in expand(state): v = max(v, min_value(s'))
  return v
def min_value(state): (symmetric, v=min, +inf)
```

Time $O(b^m)$ (b =branch factor, m =max depth); space Poly; Complete if tree finite; Optimal against optimal opponent (against suboptimal opp., minimax never does worse, but might not exploit mistakes).

Alpha-Beta Pruning

Same result as minimax, fewer nodes evaluated. α = best (highest) value for A found so far on path; β = best (lowest) value for B found so far on path. α/β = "best guaranteed option elsewhere" – stop exploring once a branch can't beat it.

```
def max_value(state,a,b):
  if terminal: return utility(state)
  v=-inf
  for s' in expand(state):
    v=max(v,min_value(s',a,b)); a=max(a,v)
    if v>=b: return v # prune
  return v
def min_value(state,a,b): (symmetric: v=min, b=min(b,v), prune if v<=a)
```

Rule: at MIN node, if $value \leq \alpha$, stop (B won't let A reach here). At MAX node, if $value \geq \beta$, stop (A won't let B improve). Best case (perfect move ordering): $O(b^{m/2})$ – doubles usable search depth.

Handling Large Game Trees

Cutoff: **is_cutoff**(state) = true if terminal OR depth/time budget hit. **Evaluation function**: terminal $\rightarrow$ utility; else $\rightarrow$ heuristic, with $min\ utility \leq heuristic(s) \leq max\ utility$. (Admissibility/consistency don't apply here.) Minimax+cutoff returns best move w.r.t. the estimate – not necessarily truly optimal. Inventing heuristics: relaxed game (e.g. material count), expert knowledge, learned from data.

6. Intro to Machine Learning

ML: subfield of AI – learn & improve performance from data. **Types:** Supervised (labeled data), Unsupervised (unlabeled, find structure), Semi-supervised, Reinforcement.

Supervised Learning

Data $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$, drawn i.i.d. from fixed distribution. **Model** $h : X \rightarrow Y$. **Loss:** guides learning (training data). **Performance measure:** judges at test time (may share form with loss). **Classification:** $y \in \{1, \dots, K\}$ (discrete). **Regression:** $y \in \mathbb{R}$ (continuous).

Performance Measures – Regression

$$MSE = \frac{1}{n} \sum_i (\hat{y}^{(i)} - y^{(i)})^2 \quad MAE = \frac{1}{n} \sum_i |\hat{y}^{(i)} - y^{(i)}|$$

Performance Measures – Classification

$$Accuracy = \frac{1}{n} \sum_i \mathbb{I}[\hat{y}^{(i)} = y^{(i)}] = \frac{TP + TN}{TP + FN + FP + TN}$$
$$P = \frac{TP}{TP + FP} \quad (\text{FP costly, e.g. spam}).$$
$$R = \frac{TP}{TP + FN} \quad (\text{FN costly, e.g. cancer}).$$

$F_1 = \frac{1}{\frac{1}{P} + \frac{1}{R}}$ (harmonic mean). *Precision: of predicted-positive, how many right? Recall: of actual-positive, how many caught? F_1 = harmonic mean, balances both. For $K > 2$: compute per-class, then average.*

Supervised Learning Pipeline

Train: Training set $\rightarrow$ Learning Algorithm A (uses Loss) $\rightarrow$ Model $h(x)$. Test: Test set $\rightarrow$ Performance Measure $\rightarrow$ check generalization.

7. Decision Trees

Inputs: categorical feature vector $x \in \{c_1, \dots, c_m\}^d$. Output: class $y \in \{1, \dots, K\}$. Tree: internal nodes = test attribute; branches = outcomes; leaves = predictions (nested if-else). **Optimal DT:** fewest nodes, consistent with data (finding it is NP-hard $\Rightarrow$ use greedy heuristic).

Entropy (uncertainty of a random var)

$$H(Y) = - \sum_{i=1}^k P(y_i) \log_2 P(y_i)$$

Binary (p pos, n neg):
 $H = -\frac{p}{p+n} \log_2 \frac{p}{p+n} - \frac{n}{p+n} \log_2 \frac{n}{p+n}$. *Entropy = how "mixed" the labels are: 0=pure, 1=maximally uncertain (50/50).*

Specific conditional entropy: $H(Y|X=x)$ = entropy of Y restricted to rows where $X = x$.
Conditional entropy: $H(Y|X) = \sum_x P(X=x) H(Y|X=x)$ – remaining uncertainty about Y after knowing X .

Information Gain:

$IG(Y; X) = H(Y) - H(Y|X)$
(uncertainty before – uncertainty after). Pick attribute with *highest* IG to split.

Decision Tree Learning (DTL)

Top-down, greedy (pick locally-best split by IG, no lookahead), recursive. `def DTL(rows, attributes, default):`
 if rows empty: return default
 if rows same class: return that class
 if attributes empty: return majority_decision(rows)
 best = choose_attribute(attributes, rows) # max IG

 tree = new tree, root=best
 for value_i of best:
 sub_i = {rows with best=value_i}
 subtree_i = DTL(sub_i, attrs-best, majority(rows))
 add subtree_i as branch labelled value_i
 return tree

Base cases: (1) no data for an attribute value $\rightarrow$ parent's majority (default); (2) all same label $\rightarrow$ that label; (3) out of attributes (noisy/insufficient data) $\rightarrow$ majority vote.

Representational Completeness: for any finite consistent dataset w/ discrete features, a DT exists that perfectly classifies it. Benefit: can fit any pattern; Drawback: can fit noise too $\rightarrow$ **overfitting**.

Pruning

- (simpler, more robust, better generalization)
- **Max depth:** cap longest root-to-leaf path.
 - **Min-sample leaves:** leaf needs $\geq k$ samples, else pruned (use majority/default instead).
 - Post-pruning (after full growth) – out of scope.

Real-World Data Prep

Continuous attrs $\rightarrow$ discretize into intervals. Missing

values: use most common value (overall or within same class), sample by probability, or drop attribute/row.

8. Linear Regression

Data $D = \{(x^{(i)}, y^{(i)})\}$, $x^{(i)} \in \mathbb{R}^d$, $y^{(i)} \in \mathbb{R}$.
Regression: predict real-valued y for new x .
Linear model ($x_0 = 1$ dummy):

$$h_w(x) = w^T x = \sum_{j=0}^d w_j x_j$$

MSE Loss (the $\frac{1}{2}$ is for convenience when differentiating):

$$J_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$$

Linear regression = linear model + MSE loss.

Normal Equation

Set gradient $\nabla J_{MSE}(w) = 0$:
$$\frac{\partial J_{MSE}}{\partial w_j} = \frac{1}{n} \sum_i (w^T x^{(i)} - y^{(i)}) x_j^{(i)} = 0$$

In matrix form ($X \in \mathbb{R}^{n \times (d+1)}$ design matrix, row $i = x^{(i)T}$):
$$X^T(Xw - Y) = 0 \Rightarrow X^T X w = X^T Y \Rightarrow$$

$$w = (X^T X)^{-1} X^T Y$$

Cost dominated by matrix inversion $O(d^3)$; fails if $X^T X$ not invertible; doesn't work for non-linear models.

Gradient Descent

Walk downhill on the loss surface, one small step at a time, always facing the steepest way down.

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w_0, w_1, \dots)}{\partial w_j} \quad (\gamma > 0: \text{learning rate})$$

Common mistake: must compute *all* partials $a = \partial J / \partial w_0, b = \partial J / \partial w_1, \dots$ using the *same/old* w first, THEN update all w_j simultaneously – not sequentially (that's coordinate descent, different algorithm, no convergence guarantee). γ too large: overshoots/diverges. γ too small: slow convergence.

Convexity

Convex: chord lies above/on graph. Strictly convex: chord always strictly above (except endpoints). Convex $\Rightarrow$ local min = global min. Strictly convex $\Rightarrow$ unique global min. **Thm:** MSE+linear model is convex; strictly convex if features linearly

independent (no feature = lin. combo of others). $\Rightarrow$ GD converges to global min (unique if lin. indep. features), given suitable γ .

Feature Scaling

Problem: different feature scales $\Rightarrow$ elongated bowl $\Rightarrow$ GD zig-zags slowly. **Min-max:**

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)} \quad (\text{scales to } [0, 1]).$$

Standardization: $x'_i = \frac{x_i - \mu_i}{\sigma_i}$ (mean 0, std 1).

Alt.: separate learning rate γ_j per weight (e.g. AdaGrad).

GD Variants

- **Batch GD:** all n examples per step. Accurate, slow/step.
- **Mini-batch GD:** subset per step. Cheaper, noisy gradient (can escape local min/plateau).
- **Stochastic GD (SGD):** 1 random example per step. Cheapest, noisiest.

Other issues: slow on large n (needs full pass); non-convex losses (future topics) can trap GD in local min/plateau.

GD vs Normal Equation

	GD	Normal Eq
Choose γ ?	Yes	No
Iterations	Many	None (closed form)
Large d ?	Fine	Slow, $O(d^3)$
Feature scaling?	May help	Not needed
Constraint	–	$X^T X$ invertible

Feature Transformation

$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^M$ ($M \geq d$). General model view:
 $\hat{h}(x) = h(\phi(x))$ (ϕ =feature map:
identity/handcrafted/learned; h =predictor, e.g. linear).

- **Polynomial** deg k : e.g. $\phi_{p2}([x_1, x_2]) \rightarrow [x_1, x_2, x_1 x_2, x_1^2, x_2^2]$ – "polynomial regression".
- **Log:** augment with $\log(x_i)$. **Exp:** augment with $\exp(x_i)$.

Why keep raw features too: model can zero out unneeded weight; needed if relation is part-linear part-nonlinear (e.g. $y = 3x + 0.5x^2$); robust fallback if transform assumption is wrong. **Thm:** linear regression + feature transform is still *convex* (it's linear regression on a new feature set).